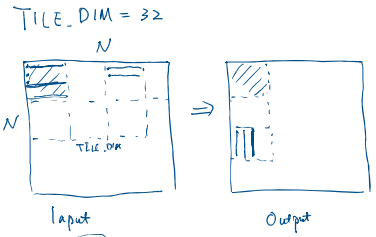
Case Study: Transpose

Tuesday, March 31, 2026 4:29 PM

```
__global__ void transposeNaive(float *odata, float* idata,
 int width, int height)
{
   int xIndex = blockIdx.x*TILE_DIM + threadIdx.x;
   int yIndex = blockIdx.y*TILE_DIM + threadIdx.y;
   int index_in = xIndex + width * yIndex;
   int index_out = yIndex + height * xIndex;
   for (int i=0; i<TILE_DIM; i+=BLOCK_ROWS) {
      odata[index_out+i] = idata[index_in+i*width];
   }
}
```

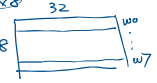
4



load idata[index_in + i * width]] Global Memory Accesses.

store odata[index_out + i]

TB: 32 x 8



TB(0,0) warp0 ⇒ blockIdx.x = 0, blockIdx.y = 0,
threadIdx.x = 0~31, threadIdx.y = 0



xIndex = 0~31
yIndex = 0
index_in = 0~31
index_out = 0, N, 2N, 3N, ..., 31N
i=0: odata[(0~31)*N] = idata[0~31];
i=8 odata[(0~31)*N+8] = idata[0~31+8*N]
i=16
i=24

Coalesced.

idata[index_in + i*width]

odata[index_out + i] — 32 # [line] accesses.

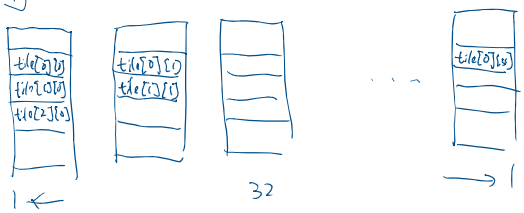
32 threads / warp → 32 x 4 elements

Sol. Shared memory to eliminate un-coalesced accesses.

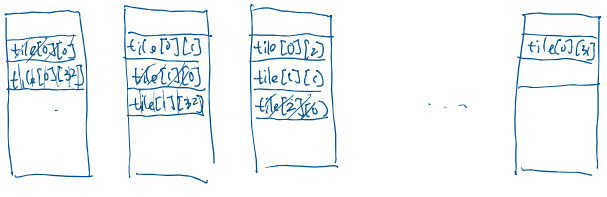
```
__global__ void transposeCoalesced(float *odata, float *idata, int width, int height)
{
   __shared__ float tile[TILE_DIM][TILE_DIM];   ⇒ tile[TILE_DIM][TILE_DIM+i];
   int xIndex = blockIdx.x*TILE_DIM + threadIdx.x;
   int yIndex = blockIdx.y*TILE_DIM + threadIdx.y;
   int index_in = xIndex + (yIndex)*width;
   xIndex = blockIdx.y * TILE_DIM + threadIdx.x;
   yIndex = blockIdx.x * TILE_DIM + threadIdx.y;
   int index_out = xIndex + (yIndex)*height;
   for (int i=0; i<TILE_DIM; i+=BLOCK_ROWS) {
      tile[threadIdx.y+i][threadIdx.x] = idata[index_in+i*width];
   }

   __syncthreads();

   for (int i=0; i<TILE_DIM; i+=BLOCK_ROWS) {
      odata[index_out+i*height] = tile[threadIdx.x][threadIdx.y+i];
   }
}
```

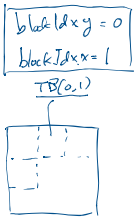


✓ tile[32][32] ⇒ tile[32][33] (Padding)



Analysis:

- Global Memory Accesses.
  - r idata[index_in+i*width]
  - w. odata[index_out+i*height] > Coalesced



TB(0,1). warp0. (warp1)

index_in for warp 0 in TB(0,1)

- back slicing (not used in this ex.)

⇓

sequential.

xIndex = 1·32 + (0~31) = 32~63

yIndex = 0 + 0 = 0

index_in = (32~63) + 0 = 32~63

xIndex = 0~31

yIndex = 32 + 0 = 32

index_out = 0~31 + 32*N

idata[index_in + i*width] ⇒ idata[32~63 + i*width] ⇒ fully coalesced.

tile[threadIdx.y+i][threadIdx.x] ⇒ tile[i][0~31] ⇒ No bank conflict.

tile[threadIdx.x][threadIdx.y+i] ⇒ tile[0~31][i] ⇒ 32-way bank conflict

Memory Partition Camping

